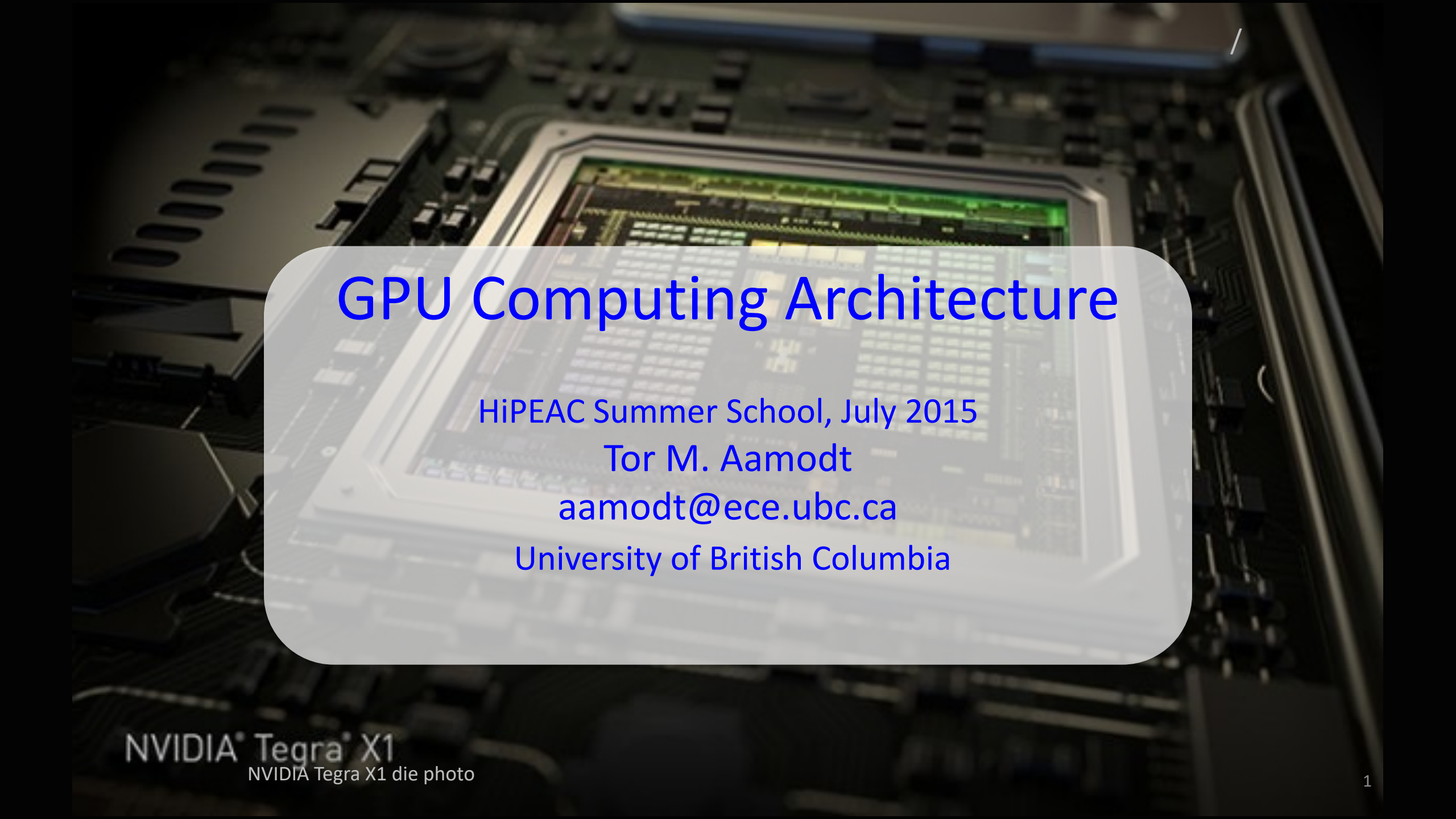


A close-up photograph of a NVIDIA Tegra X1 system-on-chip (SoC) die. The die is a square, green silicon chip with a complex grid of gold-colored bumps (Ball Grid Array) around its perimeter. It is mounted on a dark, multi-layered printed circuit board (PCB) which has various other components and traces visible. The lighting is dramatic, highlighting the metallic surfaces of the die and the intricate patterns on the PCB.

GPU Computing Architecture

HiPEAC Summer School, July 2015

Tor M. Aamodt

aamodt@ece.ubc.ca

University of British Columbia

NVIDIA® Tegra® X1

NVIDIA Tegra X1 die photo



MORGAN & CLAYPOOL PUBLISHERS

Hello. [Sign in](#) to personalize your visit. New user? [Register now](#).
[Home](#) | [Synthesis](#) | [Colloquium](#) | [Search](#) | [Profile](#) | [Author](#) | [Help](#) | [About](#) | [Bookstore](#)
Quick search: within: [go](#)

General-Purpose Graphics Processor Architectures

Synthesis Lectures on Computer Architecture

May 2018, 140 pages, (<https://doi.org/10.2200/S00848ED1V01Y201804CAC044>)

Tor M. Aamodt
University of British Columbia

Wilson Wai Lun Fung
Samsung Electronics

Timothy G. Rogers
Purdue University

Abstract

Originally developed to support video games, graphics processor units (GPUs) are now increasingly used for general-purpose (non-graphics) applications ranging from machine learning to mining of cryptographic currencies. GPUs can achieve improved performance and efficiency versus central processing units (CPUs) by dedicating a larger fraction of hardware resources to computation. In addition, their general-purpose programmability makes contemporary GPUs appealing to software developers in comparison to domain-specific accelerators. This book provides an introduction to those interested in studying the architecture of GPUs that support general-purpose computing. It collects together information currently only found among a wide range of disparate sources. The authors led development of the GPGPU-Sim simulator widely used in academic research on GPU architectures.

The first chapter of this book describes the basic hardware structure of GPUs and provides a brief overview of their history. Chapter 2 provides a summary of GPU programming models relevant to the rest of the book. Chapter 3 explores the architecture of GPU compute cores. Chapter 4 explores the architecture of the GPU memory system. After describing the architecture of existing systems, Chapters \ref{ch03} and \ref{ch04} provide an overview of related research. Chapter 5 summarizes cross-cutting research impacting both the compute core and memory system.

This book should provide a valuable resource for those wishing to understand the architecture of graphics processor units (GPUs) used for acceleration of general-purpose applications and to those who want to obtain an introduction to the rapidly growing body of research exploring how to improve the architecture of these GPUs.

Table of Contents: Preface / Acknowledgments / Introduction / Programming Model / The SIMT Core: Instruction and Register Data Flow / Memory System / Crosscutting Research on GPU Computing Architectures / Bibliography / Authors' Biographies

[PDF \(1407 KB\)](#) [PDF Plus \(1806 KB\)](#)
[Home](#) > [Series home](#) > [Abstract](#)

[Prev. lecture](#) | [Next lecture](#)
[View/Print PDF \(1407 KB\)](#)
[View PDF Plus \(1806 KB\)](#)
[Add to favorites](#)
[Email to a friend](#)
[XML](#) | [TOC Alert](#) | [Citation Alert](#)
[What is RSS?](#)

Quick Links

- [Purchase print or personal eBook](#)
- Alert me when:
[New articles cite this article](#)
- [Download to citation manager](#)
- Related articles found in:
[Morgan & Claypool](#)
- [View Most Downloaded Articles](#)

Quick Search

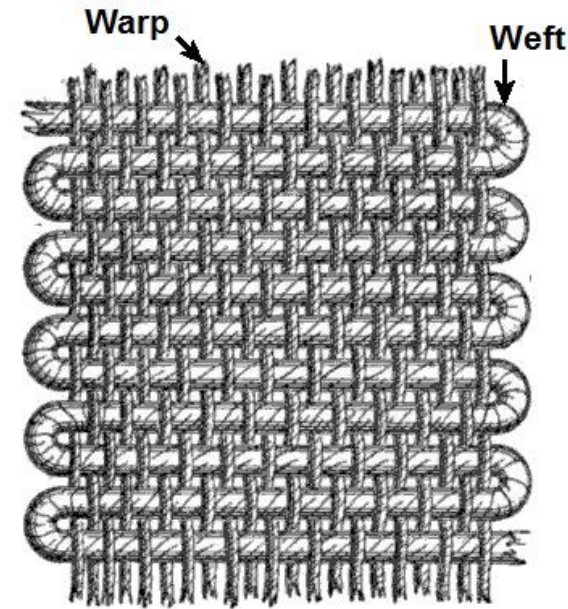
 [go](#)

Authors:

- ☐ Tor M. Aamodt
- ☐ Wilson Wai Lun Fung
- ☐ Timothy G. Rogers

SIMT Execution Model

- Programmers sees **MIMD threads** (scalar)
- GPU bundles threads into **warps** (wavefronts) and runs them in lockstep on **SIMD hardware**
- An NVIDIA warp groups 32 consecutive threads together (AMD wavefronts group 64 threads together)
- Aside: Why “Warp”? In the textile industry, the term “warp” refers to “the threads stretched lengthwise in a loom to be crossed by the weft” [Oxford Dictionary].



[https://en.wikipedia.org/wiki/Warp_and_woof]

SIMT Execution Model

- Challenge: How to handle branch operations when different threads in a warp follow a different path through program?
- Solution: Serialize different paths.

```
foo[] = {4,8,12,16};
```

```
A: v = foo[threadIdx.x];
```

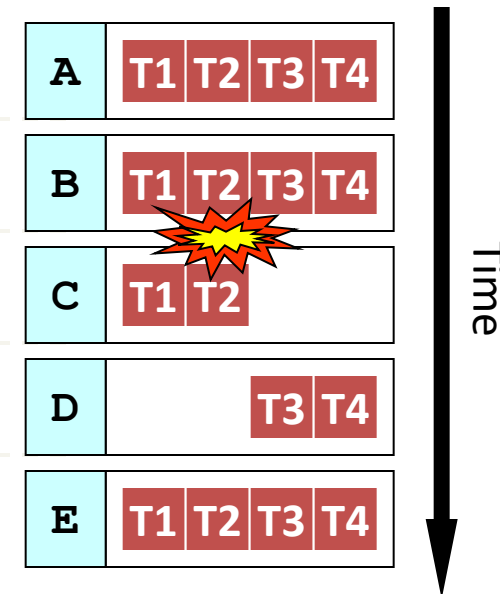
```
B: if (v < 10)
```

```
C:     v = 0;
```

```
    else
```

```
D:     v = 10;
```

```
E: w = bar[threadIdx.x]+v;
```



GPU Instruction Set Architecture (ISA)

- NVIDIA defines a virtual ISA, called “PTX” (Parallel Thread eXecution)
- More recently, Heterogeneous System Architecture (HSA) Foundation (AMD, ARM, Imagination, Mediatek, Samsung, Qualcomm, TI) defined the HSAIL virtual ISA.
- PTX is Reduced Instruction Set Architecture (e.g., load/store architecture)
- Virtual: infinite set of registers (much like a compiler intermediate representation)
- PTX translated to hardware ISA by backend compiler (“ptxas”). Either at compile time (nvcc) or at runtime (GPU driver).

Some Example PTX Syntax

- Registers declared with a type:

```
.reg .pred  p, q, r;
```

```
.reg .u16   r1, r2;
```

```
.reg .f64   f1, f2;
```

- ALU operations

```
add.u32 x, y, z;           //  $x = y + z$ 
```

```
mad.lo.s32 d, a, b, c;    //  $d = a * b + c$ 
```

- Memory operations:

```
ld.global.f32 f, [a];
```

```
ld.shared.u32 g, [b];
```

```
st.local.f64 [c], h
```

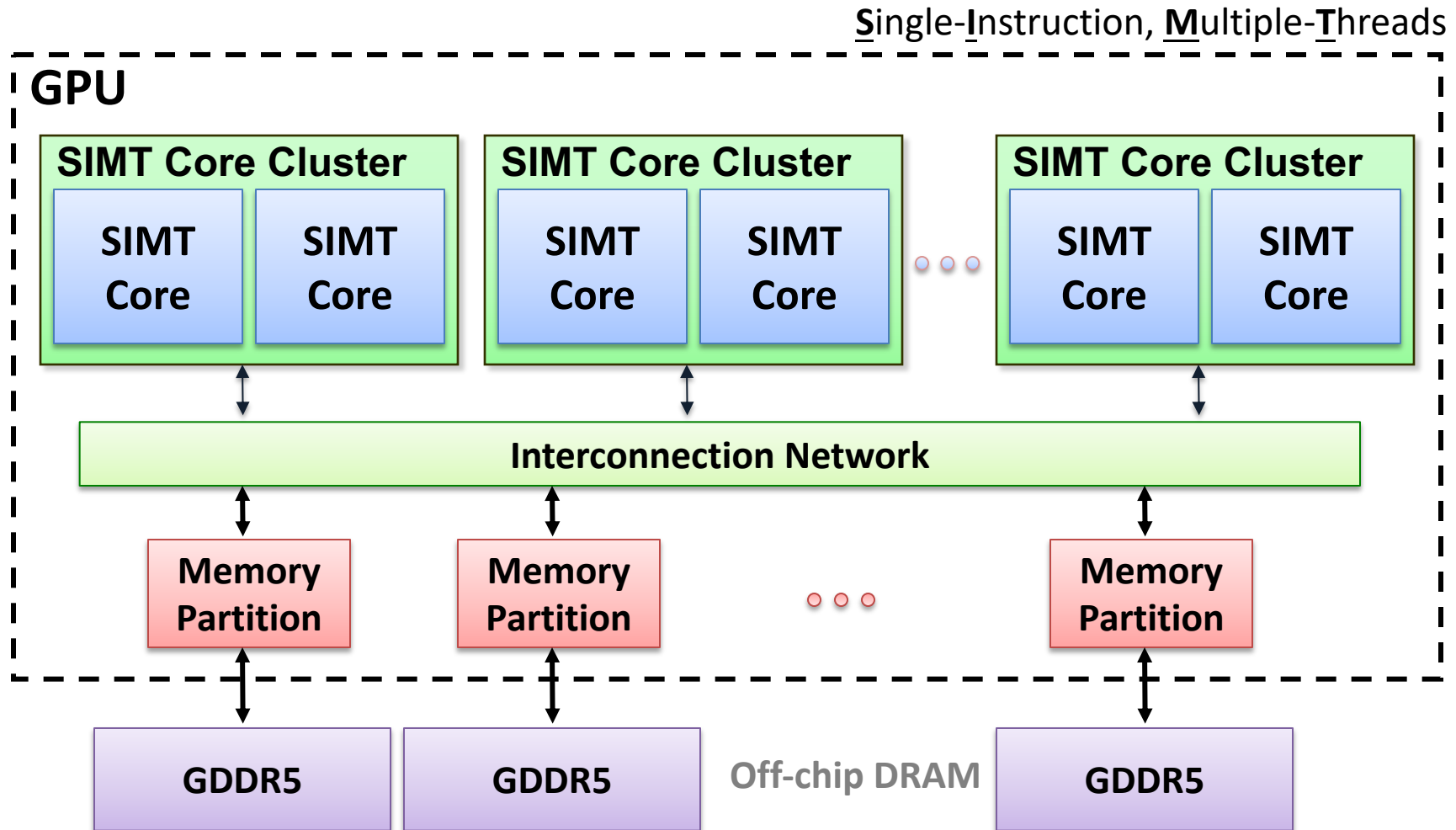
- Compare and branch operations:

```
setp.eq.f32 p, y, 0;      // is y equal to zero?
```

```
@p bra L1    // branch to L1 if y equal to zero
```

Part 2: Generic GPGPU Architecture

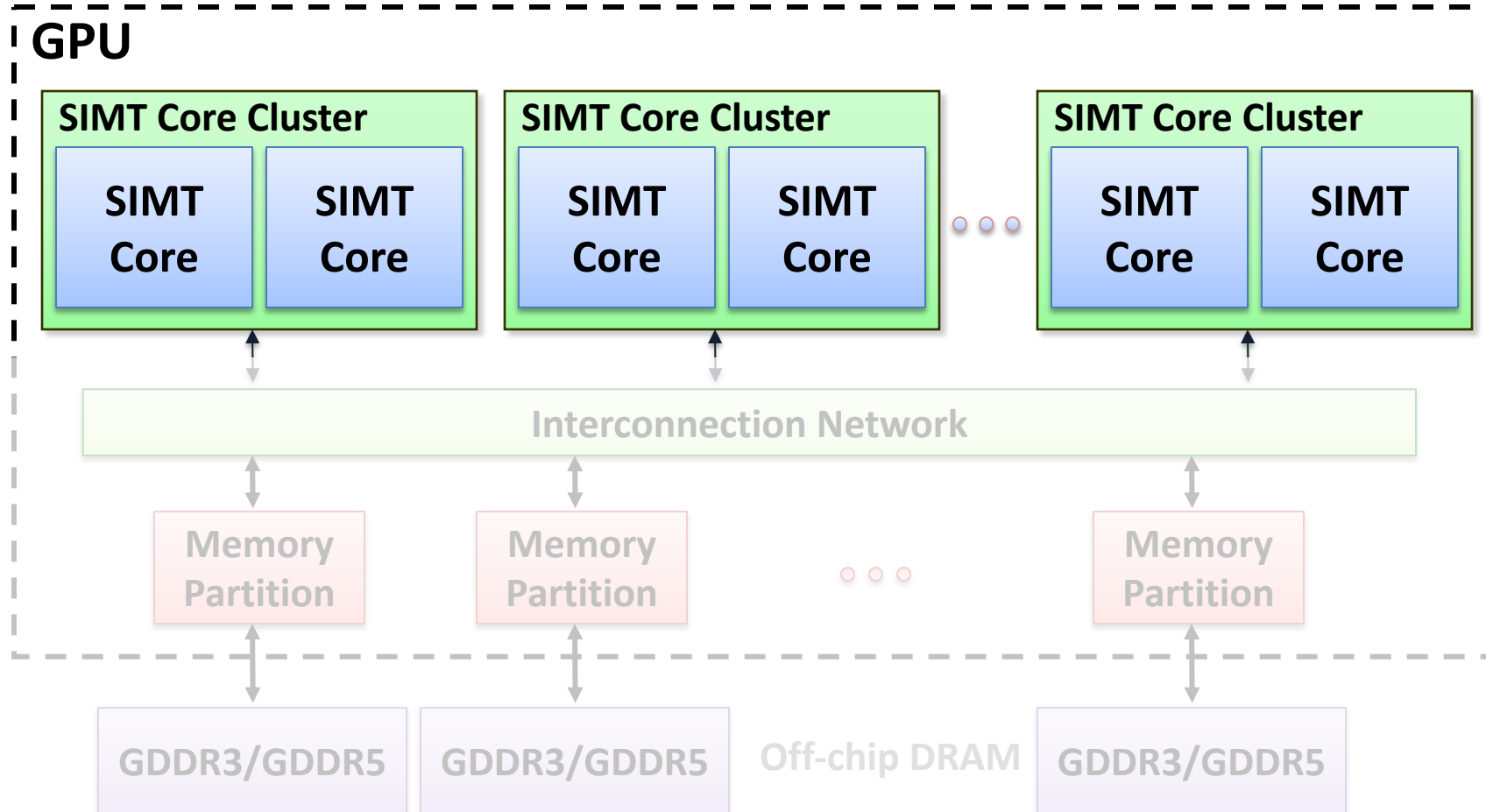
GPU Microarchitecture Overview



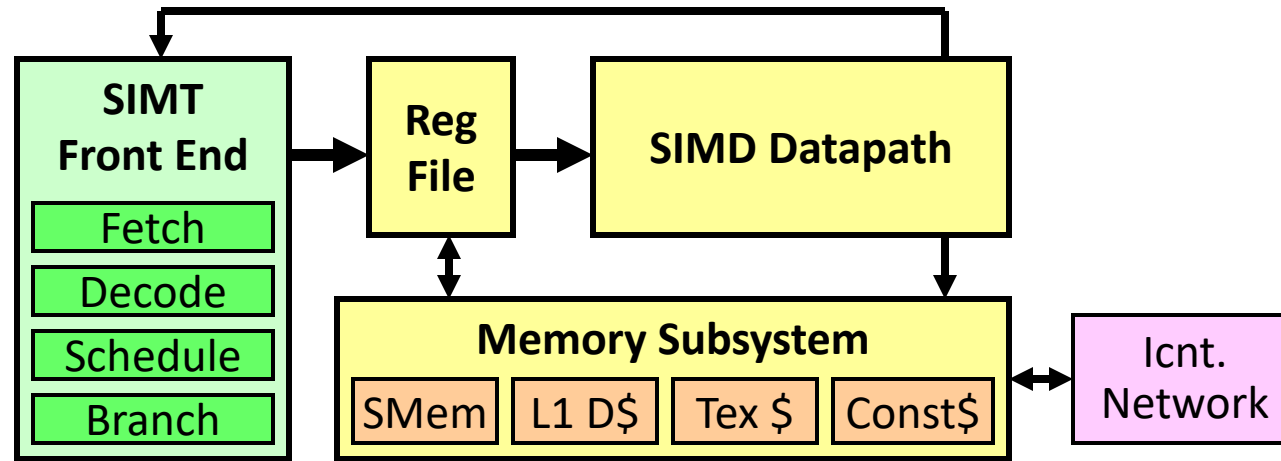
GPU Microarchitecture

- Companies tight lipped about details of GPU microarchitecture.
- Several reasons:
 - Competitive advantage
 - Fear of being sued by “non-practicing entities”
 - The people that know the details too busy building the next chip
- Model described next, embodied in GPGPU-Sim, developed from: white papers, programming manuals, IEEE Micro articles, patents.

GPU Microarchitecture Overview

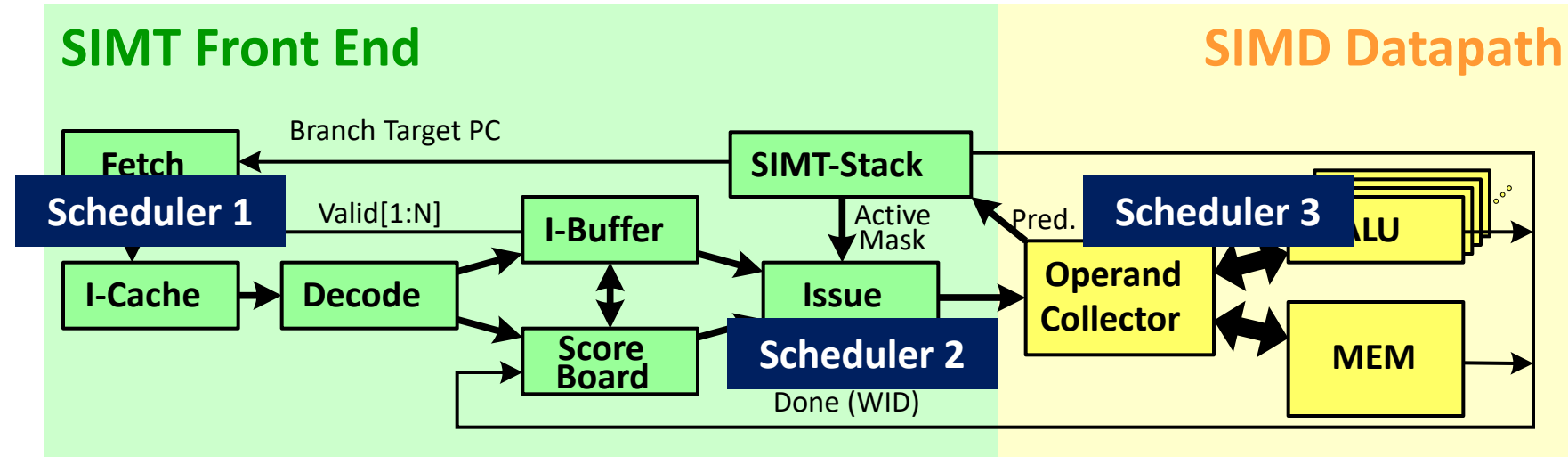


Inside a SIMT Core



- SIMT front end / SIMD backend
- Fine-grained multithreading
 - Interleave warp execution to hide latency
 - Register values of all threads stays in core

Inside an “NVIDIA-style” SIMT Core

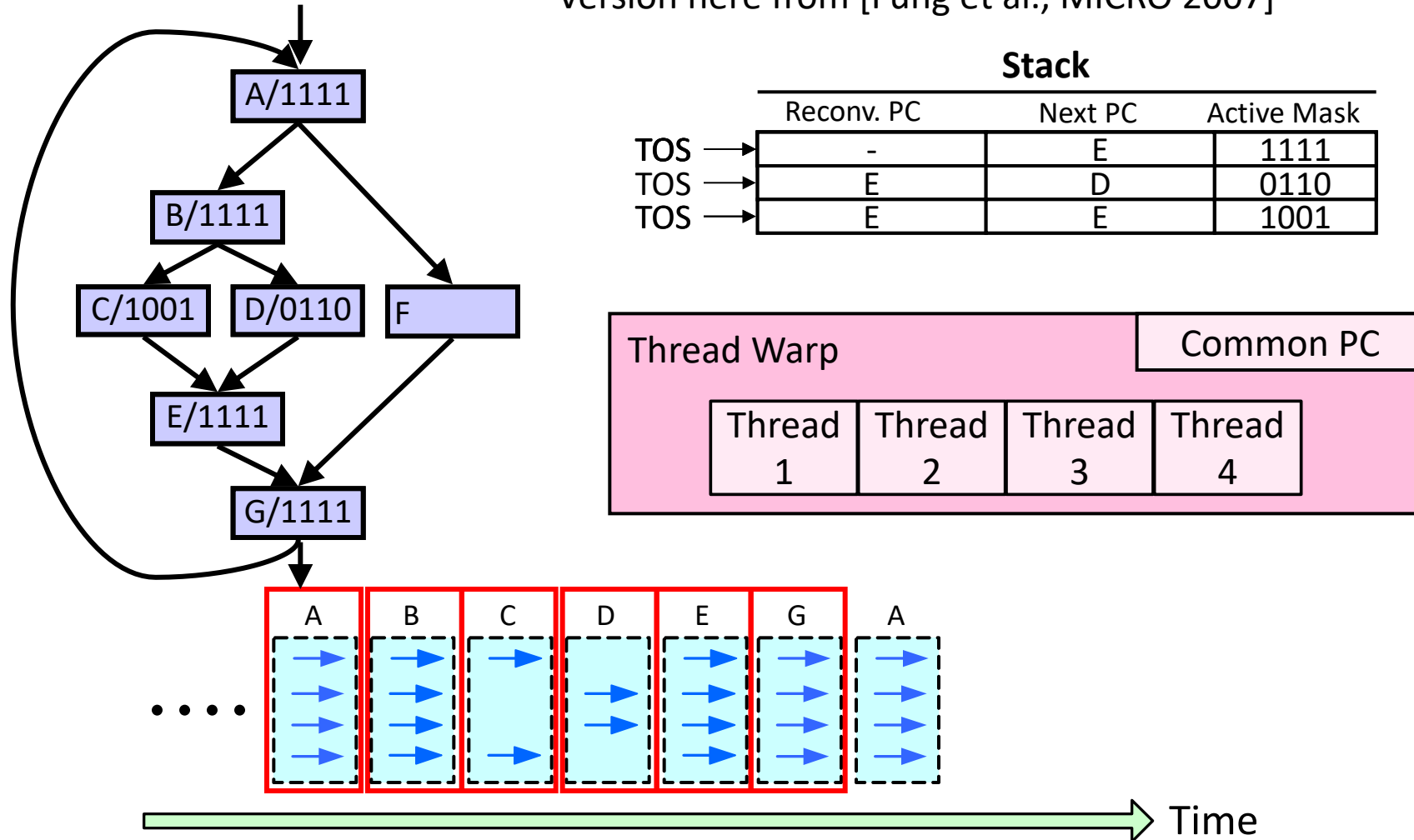


- Three decoupled warp schedulers
- Scoreboard
- Large register file
- Multiple SIMD functional units

SIMT Using a Hardware Stack

Stack approach invented at Lucasfilm, Ltd in early 1980's

Version here from [Fung et al., MICRO 2007]



SIMT = SIMD Execution of Scalar Threads